In [ ]:
```
import marimo as mo
```

Можно ли смоделировать реальный процесс одним биологическим нейроном?

Цель работы

Идея этого исследования возникла из любопытства: **можно ли взять модель биологического нейрона и попробовать с её помощью описать или спрогнозировать какой-нибудь процесс из реального мира?** Вместо того чтобы начинать с большой нейросети, мы используем модель ФицХью—Нагумо (FHN) — упрощённую математическую модель возбудимой клетки.

В качестве примера возьмём годовую активность солнечных пятен. Это не потому, что солнечная активность биологическая, а потому, что у неё есть динамика и цикличность, а исторические измерения позволяют честно проверить прогнозы на отложенных годах.

Главный вопрос: несёт ли динамика FHN-нейрона полезную информацию для прогноза временного ряда — и помогает ли настройка его параметров?

> Это эксперимент по моделированию временного ряда, а не утверждение, что Солнце буквально работает как нейрон. Результат одного набора данных нельзя автоматически переносить на общественные процессы или другие явления.

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from scipy.optimize import differential_evolution
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.preprocessing import StandardScaler
```

1. Данные и план проверки

Используем встроенный в `statsmodels` исторический ряд годовой активности солнечных пятен (1700–2008). Последние 20% наблюдений оставим для теста. Они не участвуют в подборе параметров: это имитация ситуации, когда мы обучились на прошлом и проверяемся на будущем.

Сравним три подхода:

1. **Наивный ориентир (persistence):** прогноз на следующий год равен фактическому значению текущего года. Такой ориентир важен: сложная модель полезна только если она превосходит простой разумный способ.
2. **Один настроенный FHN:** параметры нейрона подбираются на внутреннем валидационном отрезке, который находится внутри обучающего периода.
3. **Ансамбль из 48 FHN:** нейроны имеют разные, случайно выбранные параметры; затем обучается линейный слой, который читает их состояния.

Для честности все три метода оцениваются на одних и тех же тестовых годах в режиме прогноза на один шаг вперёд. Для каждого тестового года используется уже известное значение предыдущего года. Это не то же самое, что предсказать сразу следующие 10 лет без новых наблюдений.

In [ ]:
```
df = sm.datasets.sunspots.load_pandas().data
years = df["YEAR"].to_numpy(dtype=float)
y = df["SUNACTIVITY"].to_numpy(dtype=float)
split = int(len(y) * 0.8)
scaler = StandardScaler().fit(y[:split, None])
z = scaler.transform(y[:, None]).ravel()
data_summary = pd.DataFrame({
    "Показатель": ["Период данных", "Всего наблюдений", "Обучающий период",
"Тестовый период", "Доля теста"],
    "Значение": [f"{int(years[0])}–{int(years[-1])}", len(y), f"
{int(years[0])}–{int(years[split-1])} ({split})", f"{int(years[split])}–
{int(years[-1])} ({len(y)-split})", "20%"]
})
```

In [ ]:
```
mo.md("### Что содержат данные?\nСначала посмотрим на весь исторический ряд
и границу обучающей и тестовой частей. На тестовом участке модель не должна
подбирать параметры.")
fig_data, ax_data = plt.subplots(figsize=(10, 3.8))
ax_data.plot(years[:split], y[:split], label="Обучающая часть")
ax_data.plot(years[split:], y[split:], label="Отложенный тест")
ax_data.axvline(years[split], linestyle="--", color="black", alpha=0.7,
label="Граница train/test")
ax_data.set(title="Годовая активность солнечных пятен", xlabel="Год",
ylabel="Активность")
ax_data.grid(alpha=0.25)
ax_data.legend(frameon=False)
plt.tight_layout()
mo.vstack([mo.ui.table(data_summary), fig_data])
```

Search...

Показатель str	Значение object
Период данных	1700–2008
Всего наблюдений	309
Обучающий период	1700–1946 (247)
Тестовый период	1947–2008 (62)
Доля теста	20%

5 rows, 2 columns *No selection*



2. Как FHN превращается в модель временного ряда

В модели есть две переменные состояния:

- `v` — условный мембранный потенциал;
- `w` — переменная восстановления, которая влияет на динамику `v`.

Используем классические уравнения FHN:

$$\frac{dv}{dt} = v - \frac{v^3}{3} - w + I(t), \qquad \frac{dw}{dt} = \varepsilon(v + a - bw).$$

Здесь `I(t)` — внешний ток. В нашем эксперименте он зависит от нормализованного значения временного ряда: данные подаются нейрону как входной сигнал. Параметры `a`, `b`, `ε`, базовый ток и усиление входа определяют реакцию модели.

Нейрон не выдаёт прогноз сам по себе. Мы записываем его состояния `v` и `w`, а затем обучаем линейный readout (Ridge-регрессию) предсказывать следующее значение ряда по этим состояниям. Это распространённая идея резервуарного вычисления: внутренняя динамика создаёт признаки, а обучается относительно простой выходной слой.

Что именно обучается? В одиночном варианте оптимизируем пять параметров, а линейный выходной слой обучаем отдельно. В ансамбле параметры нейронов случайны и фиксированы; обучается только выходной слой. Поэтому сравнение

проверяет не «нейрон против нейронной сети вообще», а конкретные реализации этого подхода.

In [ ]:

```
DT = 0.02
SUBSTEPS = 25
N_ENSEMBLE = 48
SEED = 17
RIDGE_ALPHA = 10.0

def advance(v, w, u, params, dt=DT, substeps=SUBSTEPS):
    a, b, eps, base_current, gain = params
    current = base_current + gain * np.clip(u, -3.0, 3.0)
    v, w = v.copy(), w.copy()
    for _ in range(substeps):
        dv = v - v**3 / 3.0 - w + current
        dw = eps * (v + a - b * w)
        v += dt * dv
        w += dt * dw
        if not (np.all(np.isfinite(v)) and np.all(np.isfinite(w))):
            raise FloatingPointError("Нестабильная траектория")
        if np.max(np.abs(v)) > 20 or np.max(np.abs(w)) > 20:
            raise FloatingPointError("Траектория расходится")
    return v, w

def single_states(input_series, params):
    v, w = np.array([-1.0]), np.array([-0.5])
    states = np.zeros((len(input_series), 2))
    for t, u in enumerate(input_series):
        v, w = advance(v, w, u, params)
        states[t] = v[0], w[0]
    return states

def ensemble_states(input_series, n=N_ENSEMBLE, seed=SEED):
    rng = np.random.default_rng(seed)
    params = np.vstack([
        rng.uniform(0.55, 0.90, n), rng.uniform(0.65, 1.00, n),
        rng.uniform(0.035, 0.12, n), rng.uniform(0.25, 0.75, n),
        rng.uniform(0.08, 0.28, n),
    ]).T
    v, w = np.full(n, -1.0), np.full(n, -0.5)
    states = np.zeros((len(input_series), 2*n))
    for t, u in enumerate(input_series):
        for j in range(n):
            vj, wj = advance(v[j:j+1], w[j:j+1], u, params[j])
            v[j], w[j] = vj[0], wj[0]
        states[t, :n], states[t, n:] = v, w
    return states

def add_bias(x):
    return np.column_stack([x, np.ones(len(x))])

def rmse(actual, predicted):
    return float(np.sqrt(np.mean((np.asarray(actual) -
np.asarray(predicted))**2)))
```

3. Эксперимент A: подбираем параметры одного нейрона

Почему это делаем? Случайно выбранные параметры могут давать динамику, которая не подходит конкретному ряду. Поэтому используем дифференциальную эволюцию — алгоритм поиска параметров по качеству прогноза на валидационном отрезке.

Оптимизатор минимизирует RMSE на валидации внутри обучающего периода. Тестовый участок остаётся закрытым до финального сравнения. Чтобы время выполнения было приемлемым в интерактивном ноутбуке, число итераций оптимизации ограничено; это учебный эксперимент, а не исчерпывающий поиск глобального оптимума.

In [ ]:
```
val_start = int(split * 0.80)
bounds = [(0.3, 1.2), (0.3, 1.5), (0.01, 0.20), (-0.4, 1.4), (0.01, 1.0)]
evaluation_count = [0]

def objective(theta):
    evaluation_count[0] += 1
    try:
        states = single_states(z[:split], theta)
        X = add_bias(states[:-1])
        target = z[1:split]
        model = Ridge(alpha=RIDGE_ALPHA).fit(X[:val_start-1],
target[:val_start-1])
        pred = model.predict(X[val_start-1:split-1])
        return rmse(target[val_start-1:], pred)
    except (FloatingPointError, OverflowError, ValueError):
        return 1e4

optimization = differential_evolution(
    objective, bounds, seed=17, maxiter=5, popsize=5,
    tol=0.015, polish=True, updating="immediate", workers=1
)
tuned_params = optimization.x
parameter_table = __import__("pandas").DataFrame({
    "Параметр": ["a", "b", "ε", "Базовый ток I_ext", "Усиление входа"],
    "Подобранное значение": tuned_params
})
optimization_info = f"Вычислений целевой функции: {evaluation_count[0]}.
Валидация: {int(1700 + val_start)}–{int(1700 + split - 1)} (ориентировочные
годы; точные годы ниже определяются по данным)."
```

In [ ]:
```
mo.vstack([mo.md("### Найденные параметры одного FHN-нейрона"),
mo.ui.table(parameter_table), mo.md(optimization_info)])
```

Найденные параметры одного FHN-нейрона

Search...

Параметр str	Подобранное значение float64
a	0,30
b	1,50
ε	0,01
Базовый ток I_ext	1,40
Усиление входа	1,00

5 rows, 2 columns *No selection*

Вычислений целевой функции: 116. Валидация: 1897–1946 (ориентировочные годы; точные годы ниже определяются по данным).

In [ ]:
```python
# Recompute states for the full series with fixed, already selected parameters.
states_single = single_states(z, tuned_params)
states_ensemble = ensemble_states(z)
model_single = Ridge(alpha=RIDGE_ALPHA).fit(add_bias(states_single[:split-1]), z[1:split])
model_ensemble = Ridge(alpha=RIDGE_ALPHA).fit(add_bias(states_ensemble[:split-1]), z[1:split])

pred_single_z = model_single.predict(add_bias(states_single[split-1:-1]))
pred_ensemble_z = model_ensemble.predict(add_bias(states_ensemble[split-1:-1]))
pred_single = scaler.inverse_transform(pred_single_z.reshape(-1, 1)).ravel()
pred_ensemble = scaler.inverse_transform(pred_ensemble_z.reshape(-1, 1)).ravel()
pred_persistence = y[split-1:-1]
actual = y[split:]
test_years = years[split:]

predictions = pd.DataFrame({
    "Год": test_years, "Факт": actual,
    "Базовый прогноз": pred_persistence,
    "Один настроенный FHN": pred_single,
    f"Случайный ансамбль ({N_ENSEMBLE})": pred_ensemble,
})
metric_rows = []
for model_name, prediction in [
    ("Базовый прогноз", pred_persistence),
    ("Один настроенный FHN", pred_single),
    (f"Случайный ансамбль ({N_ENSEMBLE})", pred_ensemble),
]:
    metric_rows.append({"Модель": model_name, "RMSE": rmse(actual, prediction), "MAE": float(mean_absolute_error(actual, prediction))})
metrics = pd.DataFrame(metric_rows).sort_values("RMSE").reset_index(drop=True)
```

4. Эксперимент B: сравниваем прогнозы на отложенном периоде

Здесь уже не меняем параметры по результатам теста. Сравниваем прогнозы, построенные только по обучающему периоду, с фактическими значениями тестовых лет.

Метрики:

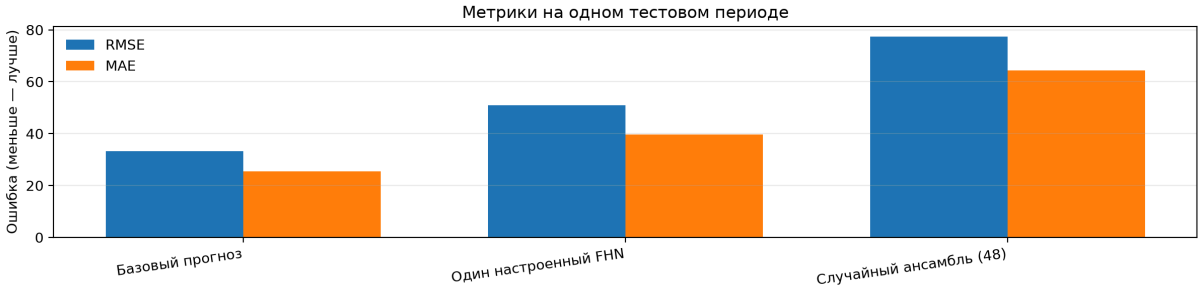
- **MAE** — средняя абсолютная ошибка; измеряется в единицах исходного ряда.
- **RMSE** — корень из средней квадратичной ошибки. Он сильнее реагирует на большие промахи.

Для обеих метрик меньше — лучше. Особенно важно сравнить FHN с базовым прогнозом: если нейронная модель хуже простого ориентира, пока нет доказательств, что её динамика помогает решать эту задачу.

In [ ]:
```
fig, axes = plt.subplots(2, 1, figsize=(12, 8), gridspec_kw=
{"height_ratios": [2.1, 1]}, constrained_layout=True)
ax = axes[0]
ax.plot(years[:split], y[:split], color="0.65", linewidth=1.0,
label="Обучающая история")
ax.plot(test_years, actual, color="black", linewidth=2.0, label="Факт на
тесте")
ax.plot(test_years, pred_persistence, linewidth=1.5, label="Базовый
прогноз")
ax.plot(test_years, pred_single, linewidth=1.5, label="Один настроенный
FHN")
ax.plot(test_years, pred_ensemble, linewidth=1.5, label="Ансамбль FHN")
ax.axvline(test_years[0], color="0.35", linestyle="--", linewidth=1.0,
label="Граница train/test")
ax.set_title("Прогноз на один год вперёд: тестовый период")
ax.set_xlabel("Год"); ax.set_ylabel("Активность солнечных пятен")
ax.grid(True, alpha=0.25); ax.legend(ncol=2, frameon=False)

ax2 = axes[1]
labels = metrics["Модель"].tolist()
rmse_values = metrics["RMSE"].to_numpy()
mae_values = metrics["MAE"].to_numpy()
x = np.arange(len(labels)); width = 0.36
ax2.bar(x - width/2, rmse_values, width, label="RMSE")
ax2.bar(x + width/2, mae_values, width, label="MAE")
ax2.set_xticks(x, labels, rotation=8, ha="right")
ax2.set_ylabel("Ошибка (меньше — лучше)")
ax2.set_title("Метрики на одном тестовом периоде")
ax2.grid(axis="y", alpha=0.25); ax2.legend(frameon=False)
plt.show()
mo.vstack([mo.md("### Численные результаты"),
mo.ui.table(metrics.round(2)), mo.md("### Примеры прогнозов на тестовых
годах"), mo.ui.table(predictions.head(15).round(2))])
```





Численные результаты

Search...

Модель str	RMSE float64	MAE float64
Базовый прогноз	33,28	25,44
Один настроенный FHN	50,85	39,59
Случайный ансамбль (48)	77,47	64,38

3 rows, 3 columns *No selection*

Примеры прогнозов на тестовых годах

Search...

Год float64	Факт float64	Базовый прогноз float64	Один настроенный FHN float64	Случайный ансамбль (48) float64
1 947	151,6	92,6	54,50	50,66
1 948	136,3	151,6	66,89	66,95
1 949	134,7	136,3	66,19	83,19
1 950	83,9	134,7	65,91	90,63
1 951	69,4	83,9	57,31	90,57
1 952	31,5	69,4	53,44	89,47
1 953	13,9	31,5	43,95	87,14
1 954	4,4	13,9	36,01	84,69
1 955	38,0	4,4	28,92	79,54
1 956	141,7	38,0	36,41	70,11

15 rows, 5 columns *No selection*

In [ ]:
```
baseline_rmse = float(metrics.loc[metrics["Модель"] == "Базовый прогноз",
"RMSE"].iloc[0])
single_rmse = float(metrics.loc[metrics["Модель"] == "Один настроенный
```

```
FHN", "RMSE"].iloc[0])
ensemble_rmse = float(metrics[metrics["Модель"].str.startswith("Случайный ансамбль")]["RMSE"].iloc[0])
if single_rmse < baseline_rmse:
    single_comment = "Один настроенный FHN превзошёл базовый прогноз по RMSE на этом тесте. Это обнадёживающий результат, но его нужно проверить на нескольких временных разбиениях."
else:
    single_comment = "Один настроенный FHN не превзошёл базовый прогноз по RMSE на этом тесте. Значит, подобранная динамика пока не даёт достаточного преимущества перед использованием последнего известного значения."
if ensemble_rmse < baseline_rmse:
    ensemble_comment = "Ансамбль превзошёл базовый прогноз по RMSE на этом тесте. Следующий шаг — проверить, сохраняется ли преимущество при других временных разбиениях и настройках."
else:
    ensemble_comment = "Случайный ансамбль не превзошёл базовый прогноз по RMSE. Разнообразие случайных нейронов само по себе не гарантирует полезных признаков; возможно, нужны другие диапазоны параметров, оптимизация или более подходящий способ подачи входа."
mo.md(f"""## 5. Как интерпретировать результат

- **Один настроенный FHN:** RMSE = **{single_rmse:.2f}**. {single_comment}
- **Случайный ансамбль:** RMSE = **{ensemble_rmse:.2f}**. {ensemble_comment}
- **Базовый прогноз:** RMSE = **{baseline_rmse:.2f}**. Он служит обязательной точкой отсчёта.

Почему нейрон может проиграть? Солнечная активность имеет выраженные циклы, а используемая модель с линейным readout может не извлекать их достаточно хорошо. Оптимизатор также ограничен по числу итераций, а ансамбль здесь намеренно не оптимизируется — его параметры случайны и фиксированы.

**Ограничения эксперимента:** один датасет, одно хронологическое разбиение, ограниченный поиск параметров и прогноз на один шаг. Поэтому результат показывает только качество именно этой реализации. Он не доказывает ни универсальную полезность, ни бесполезность биологических моделей для прогнозирования.
""")
```

5. Как интерпретировать результат

- **Один настроенный FHN:** RMSE = **50.85**. Один настроенный FHN не превзошёл базовый прогноз по RMSE на этом тесте. Значит, подобранная динамика пока не даёт достаточного преимущества перед использованием последнего известного значения.
- **Случайный ансамбль:** RMSE = **77.47**. Случайный ансамбль не превзошёл базовый прогноз по RMSE. Разнообразие случайных нейронов само по себе не гарантирует полезных признаков; возможно, нужны другие диапазоны параметров, оптимизация или более подходящий способ подачи входа.
- **Базовый прогноз:** RMSE = **33.28**. Он служит обязательной точкой отсчёта.

Почему нейрон может проиграть? Солнечная активность имеет выраженные циклы, а используемая модель с линейным readout может не извлекать их достаточно хорошо. Оптимизатор также ограничен по числу итераций, а ансамбль здесь намеренно не оптимизируется — его параметры случайны и фиксированы.
Ограничения эксперимента: один датасет, одно хронологическое разбиение, ограниченный поиск параметров и прогноз на один шаг. Поэтому результат показывает только качество именно этой реализации. Он не доказывает ни универсальную полезность, ни бесполезность биологических моделей для прогнозирования.